1. Problem Title & Link

Title: **LeetCode 209: Minimum Size Subarray Sum**

Link: https://leetcode.com/problems/minimum-size-subarray-sum/

2. Problem Statement (Short Summary)

You are given:

- A positive integer array nums
- A positive integer target

Your task is to **find the smallest length of a contiguous subarray** whose **sum is greater than or equal to target**.

If such a subarray does not exist, return **0**.

The key idea is to **find the shortest window whose sum ≥ target**.

3. Examples (Input → Output)

Example 1

Input

target = 7

nums = [2,3,1,2,4,3]

Output

2

Explanation

The smallest subarray that satisfies the condition is:

[4,3]

Sum = 7

Length = **2**

Example 2

Input

target = 4

nums = [1,4,4]

Output

1

Explanation

[4]

Length = **1**

Example 3

Input

target = 11

nums = [1,1,1,1,1,1,1,1]

Output

0

Explanation

No subarray sum reaches **11**.

4. Constraints

$1 \le target \le 10^9$

$1 \le nums.length \le 10^5$

$1 \le nums[i] \le 10^4$

Important observations:

- All numbers are **positive**
- We need **minimum window length**
- n can be **100,000**, so $O(n^2)$ will be too slow

5. Core Concept (Pattern / Topic)

Sliding Window (Variable Size Window)

Why?

Because:

- Array contains **only positive numbers**
- Window sum will **increase when expanding**
- Window sum will **decrease when shrinking**

This allows efficient two-pointer movement.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force

Try every possible subarray.

```
for i in range(n):
  sum = 0
  for j in range(i, n):
    sum += nums[j]
```

Check if:

$sum \ge target$

Time Complexity

$O(n^2)$

Too slow for n = 100000.

Step 2 — Optimization Idea

Since all numbers are **positive**, we can use **Sliding Window**.

Idea:

- Expand window to increase sum
- Shrink window to minimize length

Maintain:

window_sum

left pointer

right pointer

Step 3 — Sliding Window Strategy

Algorithm:

1. Expand right pointer
2. Add element to sum
3. If sum ≥ target
   → update minimum length
4. Shrink window from left
5. Continue

This ensures we explore **all valid windows efficiently**.

7. Visual / Intuition Diagram (ASCII)

Example

target = 7

nums = [2,3,1,2,4,3]

Window expansion

L → [2,3,1,2] ← R

sum = 8

Valid window.

Now shrink.

[3,1,2]

sum = 6

Too small.

Expand again.

[1,2,4]

sum = 7

Valid window.

Continue shrinking.

Eventually:

[4,3]

Minimum window length = **2**

8. Pseudocode (Language Independent)

```
left = 0
sum = 0
min_length = infinity

for right in range(n):

   sum += nums[right]

   while sum >= target:

      min_length = min(min_length, right-left+1)

      sum -= nums[left]

      left += 1

if min_length == infinity:
   return 0

return min_length
```

9. Code Implementation

Python

```
class Solution:
   def minSubArrayLen(self, target, nums):

      left = 0
      window_sum = 0
      min_length = float('inf')

      for right in range(len(nums)):
```

```
        window_sum += nums[right]

        while window_sum >= target:

            min_length = min(min_length, right - left + 1)

            window_sum -= nums[left]
            left += 1

    return 0 if min_length == float('inf') else min_length
```

Java

```
class Solution {
  public int minSubArrayLen(int target, int[] nums) {

    int left = 0;
    int sum = 0;
    int minLength = Integer.MAX_VALUE;

    for(int right = 0; right < nums.length; right++){

        sum += nums[right];

        while(sum >= target){

            minLength = Math.min(minLength, right-left+1);

            sum -= nums[left];
            left++;
        }
    }

    return minLength == Integer.MAX_VALUE ? 0 : minLength;
  }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Each element is visited at most **twice** (left + right pointer movement).

Space Complexity

O(1)

Only a few variables used.

11. Common Mistakes / Edge Cases

1. Forgetting to shrink window
2. Using if instead of while
3. Not handling no-solution case
4. Incorrect window length calculation
5. Forgetting positive number assumption

12. Detailed Dry Run

Example

target = 7

nums = [2,3,1,2,4,3]

Right	Value	Sum	Left	Window	Min Length
0	2	2	0	[2]	∞
1	3	5	0	[2,3]	∞
2	1	6	0	[2,3,1]	∞
3	2	8	0	[2,3,1,2]	4
		6	1	[3,1,2]	4
4	4	10	1	[3,1,2,4]	4
		7	2	[1,2,4]	3
		6	3	[2,4]	2

Answer:

2

13. Common Use Cases (Real-Life / Interview)

Sliding window is used in:

- Network traffic monitoring
- Streaming data analysis
- Real-time log processing

- Longest/shortest sequence detection
- Stock price window analysis

14. Common Traps (Important!)

1. Using if instead of while for shrinking
2. Forgetting window shrink logic
3. Missing the minimum update before shrinking
4. Assuming negative numbers exist (this method fails if negatives exist)

15. Builds To (Related LeetCode Problems)

Sliding Window progression:

LC 209 Minimum Size Subarray Sum

LC 3 Longest Substring Without Repeating Characters

LC 424 Longest Repeating Character Replacement

LC 567 Permutation in String

LC 992 Subarrays with K Different Integers

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	$O(n^2)$	Too slow
Sliding Window	$O(n)$	Optimal

17. Why This Solution Works

Because all numbers are **positive**, increasing the window increases the sum and shrinking decreases the sum. This monotonic behavior allows the **two-pointer sliding window technique** to work efficiently.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. What if numbers can be **negative**?
2. Find **maximum subarray with sum ≤ target**
3. Count **number of subarrays ≥ target**
4. Find **minimum window for product ≥ target**